

Project Design Phase Solution Architecture

Date	20 February 2026
Team ID	LTVIP2026TMIDS34731
Project Name	HouseHunt-Finding Your Prefect Rental Home
Maximum Marks	4 Marks

Solution Architecture – HouseHunt

The solution architecture of **HouseHunt** bridges the gap between traditional rental property management inefficiencies and a scalable, digital-first MERN stack web application.

The architecture is structured to serve three primary roles:

- **Renter**
- **Property Owner**
- **Admin**

The system is designed to ensure secure role-based access, modular scalability, and seamless communication between frontend and backend components.

Architectural Goals

✓ Define role-based access control and structured UI/UX flow for:

- **Renters** (Browse properties, Apply filters, Send booking requests)
- **Owners** (Add / Edit / Delete properties, Manage availability)
- **Admins** (Verify owners, Monitor users, Maintain platform integrity)

✓ Design a scalable, component-based frontend using **React.js** with responsive UI libraries (Bootstrap / Material UI), and implement role-based routing.

✓ Develop RESTful APIs using **Express.js** to enable secure communication between frontend and backend.

✓ Integrate **MongoDB** database with well-defined Mongoose schemas for:

- Users
- Properties
- Booking Requests
- Role management

✅ Implement **JWT-based authentication** with middleware for secure, verified access control.

✅ Support core modules such as:

- Property management
 - Booking workflow
 - Status tracking
 - Admin approval system
 - Dashboard analytics
-



Technical Stack Overview

Frontend Layer

- React.js
- Bootstrap / Material UI
- Axios (API communication)
- Role-based routing
- Responsive design

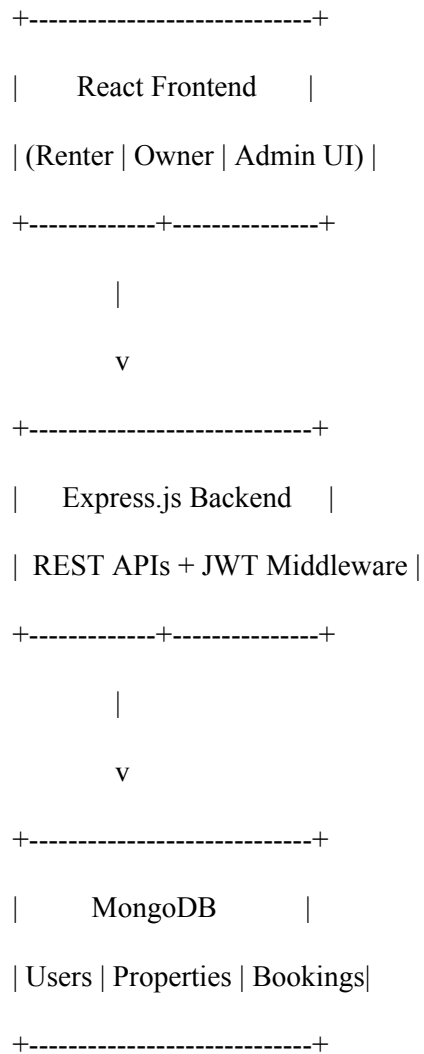
Backend Layer

- Node.js
- Express.js
- REST APIs

- Middleware for authentication & authorization

Database Layer

- MongoDB
- Mongoose Schemas:
 - Users (Renter / Owner / Admin)
 - Properties
 - Booking Requests
 -



Tech Stack Breakdown

Layer	Technology Used	Description
Frontend	React.js + React Router DOM	Dynamic UI, routing, role-based dashboard views
Backend	Node.js + Express.js	API server, middleware handling, session control
Database	MongoDB + Mongoose	Stores users, books, orders, and authentication data
Security	JWT + Role-based Access Middleware	Secure token-based login for all roles (User, Seller, Admin)